

PSTAT 231-Assignment2

Xinyi Yang

April 26th 2026

Setup

```
1 library(tidyverse)
2 library(tidymodels)
3
4 # Set seed for reproducibility
5 set.seed(42)
6
7 # Load data
8 abalone <- read_csv("data/abalone.csv")
9 glimpse(abalone)
```

Listing 1: Package loading and data import

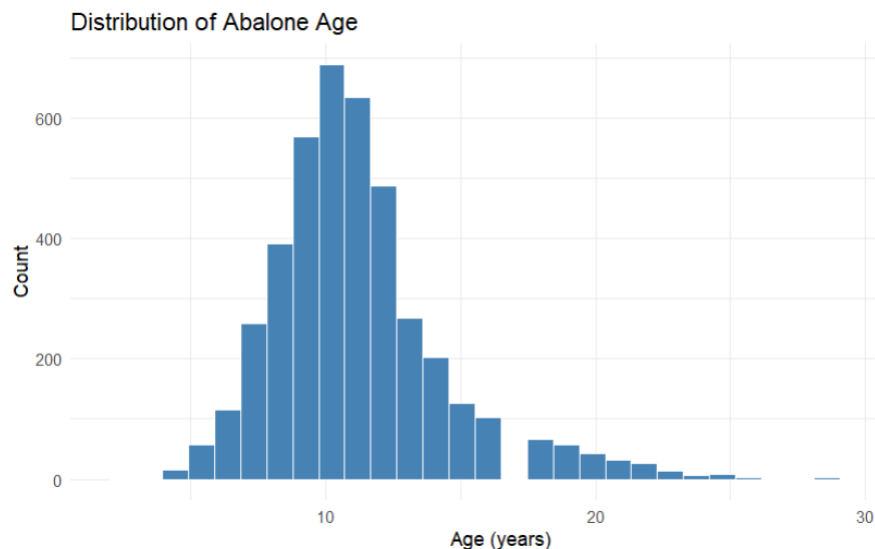
The abalone dataset contains 4,177 observations and 9 variables: `type` (M/F/I), six continuous physical measurements (`longest_shell`, `diameter`, `height`, `whole_weight`, `shucked_weight`, `viscera_weight`, `shell_weight`), and `rings` (integer). Age is defined as `rings+1.5` (in years).

Question 1

```
1 library(dplyr)
2 library(ggplot2)
3 abalone <- read_csv("C:/Users/YXY/Desktop/homework-2/data/abalone.
4   csv")
5 abalone <- abalone %>%
6   mutate(age = rings + 1.5)
7
8 summary(abalone$age)
9
10 ggplot(abalone, aes(x = age)) +
11   geom_histogram(bins = 30, fill = "steelblue", color = "white") +
12   labs(
13     title = "Distribution of Abalone Age",
14     x      = "Age (years)",
15     y      = "Count"
16   ) +
```

```
16 theme_minimal()
```

Listing 2: Add age variable and plot its distribution



The distribution of `age` is *right-skewed* (positively skewed). The majority of observations cluster between approximately 7 and 14 years, with the mode near 10–11 years. A long right tail extends beyond 20 years, with a small number of very old abalones reaching 30 years or more. The median age is approximately 10.5 years and the mean is slightly higher due to the right skew, around 11.4 years. This skewness is typical for biological age distributions in wild populations, where younger organisms are more prevalent than older ones.

Question 2

```
1 set.seed(131)
2
3 data_split <- data2 %>% initial_split(strata = age, prop = 3/4)
4
5 train <- training(data_split)
6 test  <- testing(data_split)
7
8 cat("Training set size:", nrow(train), "\n")
9 cat("Testing set size: ", nrow(test),  "\n")
```

Listing 3: Stratified train/test split

I use a 75/25 split (approximately 3,132 training observations and 1,045 testing observations), with `set.seed(131)` for reproducibility. Stratified sampling on `age` is used to ensure that the distribution of age is preserved in both the training and testing sets. This is important because `age` is right-skewed; without stratification, random chance could produce splits with noticeably different age distributions, which would compromise the validity of model evaluation.

Question 3

```
1 train_recipe <- recipe(age ~ ., data = train) %>%
2   step_rm(rings) %>%
3   step_dummy(all_nominal_predictors()) %>%
4   step_interact(
5     term = ~ starts_with("type"):shucked_weight +
6             longest_shell:diameter +
7             shucked_weight:shell_weight) %>%
8   step_normalize(all_predictors())
```

Listing 4: Recipe with dummy coding, interactions, and normalization

The variable `age` is defined deterministically as `age = rings + 1.5`. Including `rings` as a predictor would therefore constitute data leakage: the model would trivially learn this identity function and achieve near-perfect predictions on training data, but this tells us nothing useful about whether the *physical measurements* can predict age. The scientific question is whether we can *avoid* the costly microscopy step (counting rings) by using easier physical measurements. Including `rings` defeats that purpose entirely.

Question 4

```
1 lm_model <- linear_reg() %>%
2   set_engine("lm") %>%
3   set_mode("regression")
```

Listing 5: Define linear regression model

Specify ordinary least squares linear regression using the "lm" engine (base R's `lm()` function). The mode is "regression" since `age` is a continuous response variable.

Question 5

```
1 knn_model <- nearest_neighbor(neighbors = 7) %>%
2   set_engine("kkn") %>%
3   set_mode("regression")
```

Listing 6: Define KNN model with $k = 7$

We specify a K-Nearest Neighbours regression model using the "kkn" engine with $k = 7$ neighbours. KNN is a non-parametric, instance-based learner: it predicts a new observation's age by averaging the ages of its 7 nearest neighbours in the feature space (after centering and scaling, so all predictors are on the same scale).

Question 6

```

1 lm_workflow <- workflow() %>%
2   add_recipe(train_recipe) %>%
3   add_model(lm_model)
4
5 knn_workflow <- workflow() %>%
6   add_recipe(train_recipe) %>%
7   add_model(knn_model)
8
9 fit_lm <- fit(lm_workflow, data = train)
10 fit_knn <- fit(knn_workflow, data = train)

```

Listing 7: Create workflows and fit both models to training data

Each `workflow` bundles the pre-processing recipe and the model specification together, ensuring that the same transformations (dummy coding, interactions, centering, scaling) are applied consistently during both training and prediction. Both models are then fitted to `abalone_train`.

Question 7

```

1 new_data <- tibble(
2   type           = "F",
3   longest_shell  = 0.50,
4   diameter       = 0.10,
5   height         = 0.30,
6   whole_weight   = 4,
7   shucked_weight = 1,
8   viscera_weight = 2,
9   shell_weight   = 1,
10  rings          = 0      # placeholder; removed by recipe
11 )
12
13 predict(fit_lm, new_data)
14
15 ## # A tibble: 1 x 1
16 ##   .pred
17 ##   <dbl>
18 ## 1    23.8

```

Listing 8: Predict age for a hypothetical female abalone

The linear regression model predicts an age of **23.8 years** for this hypothetical female abalone. This is notably high relative to the typical range in the dataset (most abalones are 8–13 years old), but is consistent with the unusually large `whole_weight` (4 g) — far above the dataset average of ~ 0.83 g — which strongly signals a large, old specimen. The recipe removes `rings` before fitting, so the placeholder value of 0 has no effect on the prediction.

Question 8

```

1 metric <- yardstick::metric_set(rmse, rsq, mae)
2 testing_results_lm <- augment(fit_lm, test)
3
4 metric(testing_results_lm, truth = age, estimate = .pred)
5
6 ## # A tibble: 3 x 3
7 ##   .metric .estimator .estimate
8 ##   <chr>   <chr>      <dbl>
9 ## 1 rmse    standard    2.14
10 ## 2 rsq     standard    0.556
11 ## 3 mae     standard    1.55
12
13 testing_results_knn <- augment(fit_knn, test)
14
15 metric(testing_results_knn, truth = age, estimate = .pred)
16
17 ## # A tibble: 3 x 3
18 ##   .metric .estimator .estimate
19 ##   <chr>   <chr>      <dbl>
20 ## 1 rmse    standard    2.35
21 ## 2 rsq     standard    0.478
22 ## 3 mae     standard    1.69

```

Listing 9: Evaluate both models on test data

Results

Model	R^2	RMSE	MAE
Linear Regression	0.556	2.14	1.55
KNN ($k = 7$)	0.478	2.35	1.69

Interpretation of R^2

The R^2 (coefficient of determination) measures the proportion of variance in `age` that is explained by the model's predictions.

- For the linear regression model, $R^2 = 0.556$ means that approximately 55.6% of the total variation in abalone age is explained by the physical predictors (after dummy coding, interactions, centering, and scaling). The remaining $\sim 44.4\%$ is unexplained, attributable to biological variability, measurement error, and residual non-linearity.
- For the KNN model, $R^2 = 0.478$ is noticeably lower than the linear regression model, meaning KNN explains only about 47.8% of the variance in age on the test set. On average, KNN predictions deviate from the true age by 1.69 years (MAE), compared to 1.55 years for linear regression.

- Both values are moderate. Predicting abalone age from physical measurements alone is inherently difficult due to biological noise — ring counting via microscopy itself has measurement uncertainty, and physical size is an imperfect proxy for age.

Question 9

The linear regression model performed better on the testing data across all three metrics: higher R^2 (0.556 vs. 0.478), lower RMSE (2.14 vs. 2.35), and lower MAE (1.55 vs. 1.69).

1. Approximately linear structure: The relationship between physical measurements (size, weight) and abalone age changes gradually and smoothly. Physical growth in marine molluscs is relatively monotone — larger, heavier shells tend to be older — which is well-captured by a linear model. KNN performs best when data has strong local clusters or sharp non-linear boundaries, which are not prominent features of this dataset.
2. Interaction terms help the linear model: The recipe includes three interaction terms, which allow the linear model to capture some non-linearity and cross-variable effects that would otherwise be missed.
3. KNN is sensitive to the choice of k : With $k = 7$ and no hyperparameter tuning, KNN may not be using its optimal number of neighbours. A grid search over k could improve KNN performance, potentially closing the gap with linear regression.
4. Curse of dimensionality: After dummy coding and creating interaction features, the feature space has higher dimensionality. Distance-based methods like KNN become less reliable in higher dimensions, as all points tend to become equidistant, degrading neighbourhood quality.

These results are not surprising upon reflection. While it is tempting to assume that a flexible non-parametric model like KNN should outperform a linear model, the performance of any model ultimately depends on whether its assumptions match the true data-generating process. In this case, the smooth, approximately linear relationship between physical measurements and abalone age favours linear regression. The result reinforces a key lesson in machine learning: a simpler, well-specified model can outperform a more flexible one when the added flexibility is not needed — and when hyperparameters (like k) have not been optimised.

Question 10

Reducible error

$$\text{Var}\left(\hat{f}(x_0)\right) + \left[\text{Bias}\left(\hat{f}(x_0)\right)\right]^2$$

These two terms arise from the imperfection of our estimate $\hat{f}$. The **variance** term measures how much $\hat{f}$ fluctuates across different training sets (sensitivity to the particular sample used). The **bias** term measures systematic error introduced by modeling assumptions (e.g.,

assuming linearity when the true f is non-linear). Both can, in principle, be reduced by choosing a better model class, collecting more data, or tuning hyperparameters.

Irreducible error

$$\text{Var}(\varepsilon) = \sigma_\varepsilon^2$$

This term arises from the inherent noise ε in the data-generating process. No matter how perfect our estimator $\hat{f}$ is, this error cannot be eliminated because it reflects genuine randomness in Y that is not a function of X .

Question 11

Claim: The expected test error is always at least as large as the irreducible error, i.e.,

$$E[(y_0 - \hat{f}(x_0))^2] \geq \text{Var}(\varepsilon).$$

Demonstration:

From the bias-variance decomposition,

$$E[(y_0 - \hat{f}(x_0))^2] = \underbrace{\text{Var}(\hat{f}(x_0))}_{\geq 0} + \underbrace{[\text{Bias}(\hat{f}(x_0))]^2}_{\geq 0} + \text{Var}(\varepsilon).$$

Since variance and squared bias are both *non-negative* by definition (variance is an expectation of a squared quantity; the square of any real number is non-negative), their sum satisfies

$$\text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 \geq 0.$$

Therefore,

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\varepsilon) \geq 0 + \text{Var}(\varepsilon) = \text{Var}(\varepsilon). \quad \square$$

Equality holds if and only if both the variance and the bias of $\hat{f}$ are zero simultaneously (i.e., $\hat{f}(x_0) = f(x_0)$ with probability 1 for every training set), which is generally unachievable in practice.

Question 12

Claim:

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \sigma_\varepsilon^2.$$

Proof:

Let $\mu := E[\hat{f}(x_0)]$ and write $y_0 = f(x_0) + \varepsilon$ where $E[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma_\varepsilon^2$, and ε is independent of $\hat{f}(x_0)$ (since ε is the test noise, independent of the training set). Also note $\text{Bias}(\hat{f}(x_0)) = \mu - f(x_0)$.

Step 1. Add and subtract $\mu = E[\hat{f}(x_0)]$:

$$\begin{aligned} y_0 - \hat{f}(x_0) &= [f(x_0) + \varepsilon] - \hat{f}(x_0) \\ &= [f(x_0) - \mu] + [\mu - \hat{f}(x_0)] + \varepsilon. \end{aligned} \quad (1)$$

Step 2. Square both sides and take expectations. Denote $A = f(x_0) - \mu = -\text{Bias}(\hat{f}(x_0))$, $B = \mu - \hat{f}(x_0)$, $C = \varepsilon$.

$$(y_0 - \hat{f}(x_0))^2 = (A + B + C)^2 = A^2 + B^2 + C^2 + 2AB + 2AC + 2BC.$$

Take expectations:

$$E[(y_0 - \hat{f}(x_0))^2] = E[A^2] + E[B^2] + E[C^2] + 2E[AB] + 2E[AC] + 2E[BC]. \quad (2)$$

Step 3. Evaluate each term:

- $E[A^2] = (f(x_0) - \mu)^2 = [\text{Bias}(\hat{f}(x_0))]^2$ (constant in the training-set randomness).
- $E[B^2] = E[(\mu - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0))$ (since $E[\hat{f}(x_0)] = \mu$).
- $E[C^2] = E[\varepsilon^2] = \sigma_\varepsilon^2$ (since $E[\varepsilon] = 0$).
- $E[AB] = E[(f(x_0) - \mu)(\mu - \hat{f}(x_0))]$. Since $f(x_0)$ and μ are constants, $E[\mu - \hat{f}(x_0)] = \mu - \mu = 0$, so $E[AB] = 0$.
- $E[AC] = (f(x_0) - \mu) E[\varepsilon] = 0$ (since $E[\varepsilon] = 0$).
- $E[BC] = E[(\mu - \hat{f}(x_0))\varepsilon] = E[\mu - \hat{f}(x_0)] \cdot E[\varepsilon] = 0 \cdot 0 = 0$,
where the factorisation uses independence of ε and $\hat{f}(x_0)$.

Step 4. Substitute back into (2):

$$E[(y_0 - \hat{f}(x_0))^2] = [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\hat{f}(x_0)) + \sigma_\varepsilon^2 + 0 + 0 + 0.$$

Rearranging:

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\varepsilon).$$